

Spring IoC Container & Dependency Injection

Chapter 03 - Sub-Chapter 01

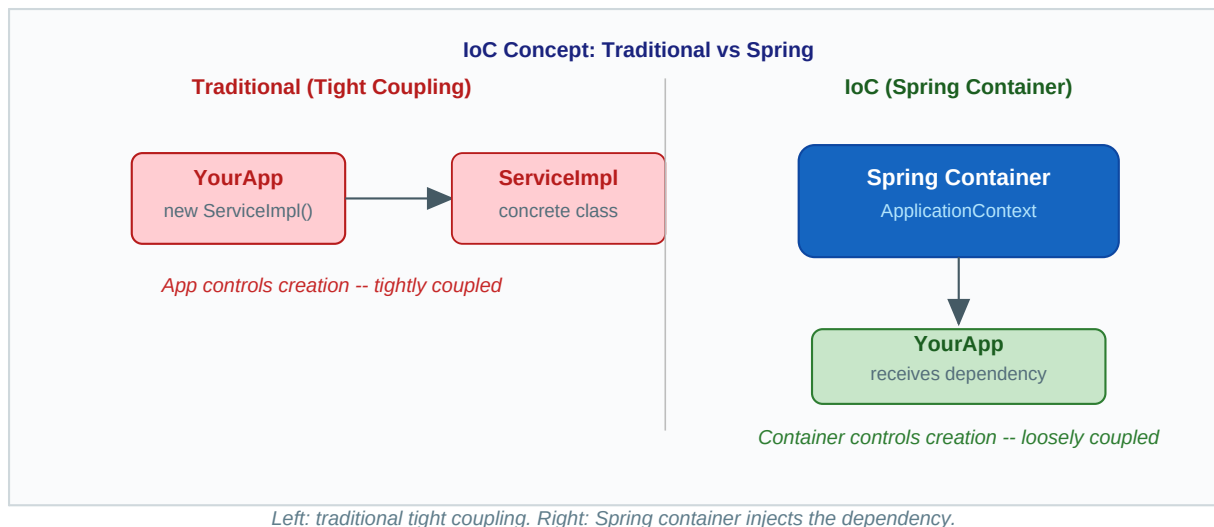
Spring Framework Series

IoC Concept	Container manages object creation & wiring
Spring Container	ApplicationContext vs BeanFactory
Injection Types	Constructor, Setter, Field -- compared
Stereotypes	@Component, @Service, @Repository, @Controller
@Autowired & friends	@Qualifier and @Primary resolve ambiguity
Java Config	@Configuration and @Bean factory methods

1 What is IoC?

Inversion of Control - Don't call us, we'll call you

Inversion of Control (IoC) is a design principle where the control of object creation and dependency wiring is handed over to a framework instead of the application code. Normally, your class creates its dependencies with **new**. With IoC, the container creates and injects them for you -- you just declare what you need.



Simple Java Example -- Without IoC (tight coupling):

```
// No Spring import needed -- plain Java
public class OrderService {
    // App creates dependency directly -- tightly coupled
    private PaymentService payment = new PaymentServiceImpl();

    public void placeOrder() {
        payment.charge(100);
    }
}
```

Simple Java Example -- With IoC (Spring injects):

```
import org.springframework.stereotype.Service;

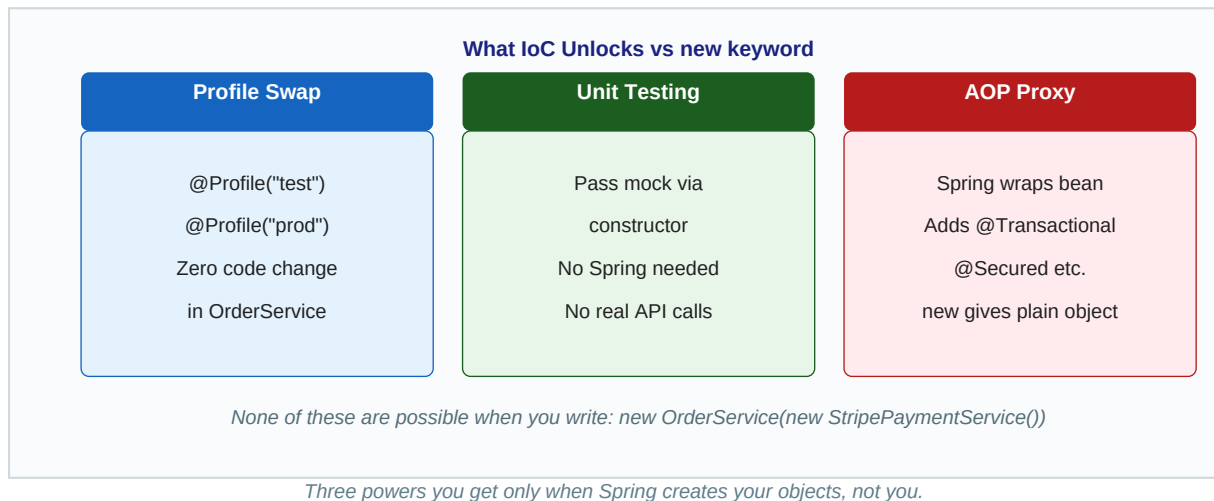
@Service
public class OrderService {
    private final PaymentService payment;

    // Spring injects PaymentService -- no 'new' needed
    public OrderService(PaymentService payment) {
        this.payment = payment;
    }

    public void placeOrder() { payment.charge(100); }
}
```

What IoC can do that new literally cannot:

Writing **new StripePaymentService()** hardwires a concrete class inside your code forever. Spring IoC, by owning object creation, unlocks three things that are impossible with **new**:



Power 1 -- Swap implementations without touching OrderService:

Scenario: In tests use a fake payment service (no real Stripe API call). In prod use the real one. With **new**, you must edit OrderService. With IoC, you change only the config:

```
import org.springframework.stereotype.Service;
import org.springframework.context.annotation.Primary;
import org.springframework.context.annotation.Profile;

public interface PaymentService { void charge(int amount); }

@Profile("prod") @Primary @Service
public class StripePaymentService implements PaymentService {
    public void charge(int amount) { /* calls Stripe API */ }
}

@Profile("test") @Primary @Service
public class FakePaymentService implements PaymentService {
    public void charge(int amount) {
        System.out.println("FAKE: charged " + amount); // no real API
    }
}

@Service
public class OrderService {
    private final PaymentService payment;
    public OrderService(PaymentService payment) { // Spring injects
        this.payment = payment;
    }
    public void placeOrder() { payment.charge(100); }
}

// Run with --spring.profiles.active=test -> FakePaymentService injected
// Run with --spring.profiles.active=prod -> StripePaymentService injected
// OrderService itself is NEVER changed
```

Power 2 -- Unit test without Spring, without real APIs:

Because the dependency arrives via the constructor, you can pass a mock directly in a plain Java test. No Spring context needed, no network calls, test runs in milliseconds:

```
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

@Test
void testPlaceOrder() {
```

```

// Create a mock -- no Spring container, no Stripe, no DB
PaymentService mock = Mockito.mock(PaymentService.class);

// Inject it directly through the constructor
OrderService service = new OrderService(mock);
service.placeOrder();

// Assert Spring would have wired the correct call
Mockito.verify(mock).charge(100);
}
// With 'new StripePaymentService()' inside OrderService, this is impossible.
// You would need reflection hacks to swap the private field in the test.

```

Power 3 -- AOP Proxy (the one new can NEVER replicate):

This is the most important difference. When Spring creates your bean, it may hand you a **generated proxy subclass** instead of your plain class. That proxy adds cross-cutting behaviour transparently -- you write none of it:

```

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class OrderService {
    @Transactional // tells Spring: wrap this method in a DB transaction
    public void placeOrder() {
        payment.charge(100);
        orderRepo.save(order); // if this throws, charge is rolled back
    }
}

// What Spring actually gives you when you call ctx.getBean(OrderService.class):
//
// OrderService$$SpringCGLIB$$0 <-- a generated subclass (proxy)
// -> beginTransaction()
// -> calls YOUR placeOrder()
// -> commit() OR rollback() on exception
//
// @Transactional does NOTHING if you write:
// OrderService svc = new OrderService(...);
// svc.placeOrder(); // plain object, no proxy, no transaction safety
//
// Same applies to @Secured, @Async, @Cacheable, @Retryable --
// all require Spring to create the object so it can wrap it.

```

One-line summary:

new gives you an object. IoC gives you a **managed** object -- swappable by config, testable with mocks, and potentially wrapped with invisible superpowers like transactions, security, and caching that activate only because Spring created it.

Key Rules

- IoC means the framework calls your code, not the other way around.
- Profile swap: change which bean Spring injects without touching the class that uses it.
- Unit testing: pass a mock via constructor -- no Spring, no real APIs needed.
- AOP proxy: @Transactional, @Secured, @Cacheable only work on Spring-managed beans.
- new gives you a plain object; Spring gives you a managed (and possibly proxied) object.

- The principle: 'Don't call us (framework), we'll call you.'

Q1. What does IoC stand for?

Inversion of Control -- a principle where object creation and wiring is delegated to a container, not the class itself.

Q2. What problem does IoC solve?

It removes tight coupling. Without IoC, classes create their own dependencies using 'new', making them hard to test or swap.

Q3. Can I swap implementations with 'new'?

No. 'new ConcreteClass()' is hardwired. You must edit every class that uses it. With IoC, you swap the bean definition in config -- the consuming class never changes.

Q4. Why does @Transactional not work with 'new'?

@Transactional is applied by a BeanPostProcessor that wraps the bean in a CGLIB proxy. If you create the object with 'new', Spring never wraps it, so the annotation does nothing.

Q5. How does IoC help unit testing?

Dependencies come in via constructor or setter, so you can pass a mock directly in a test -- no Spring container, no network, no DB needed.

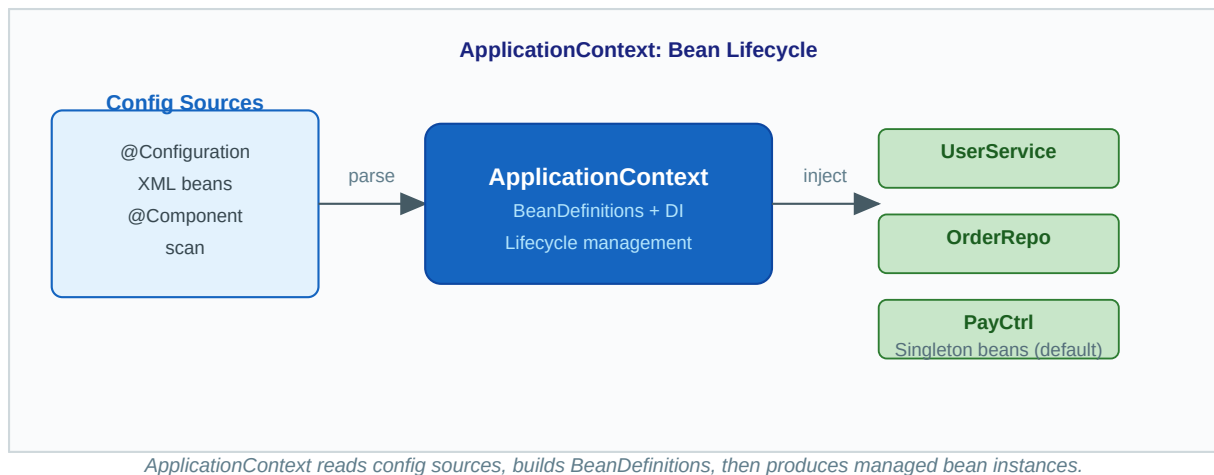
Q6. Is IoC the same as Dependency Injection?

IoC is the principle; Dependency Injection (DI) is one way to implement it. Spring uses DI as its IoC mechanism.

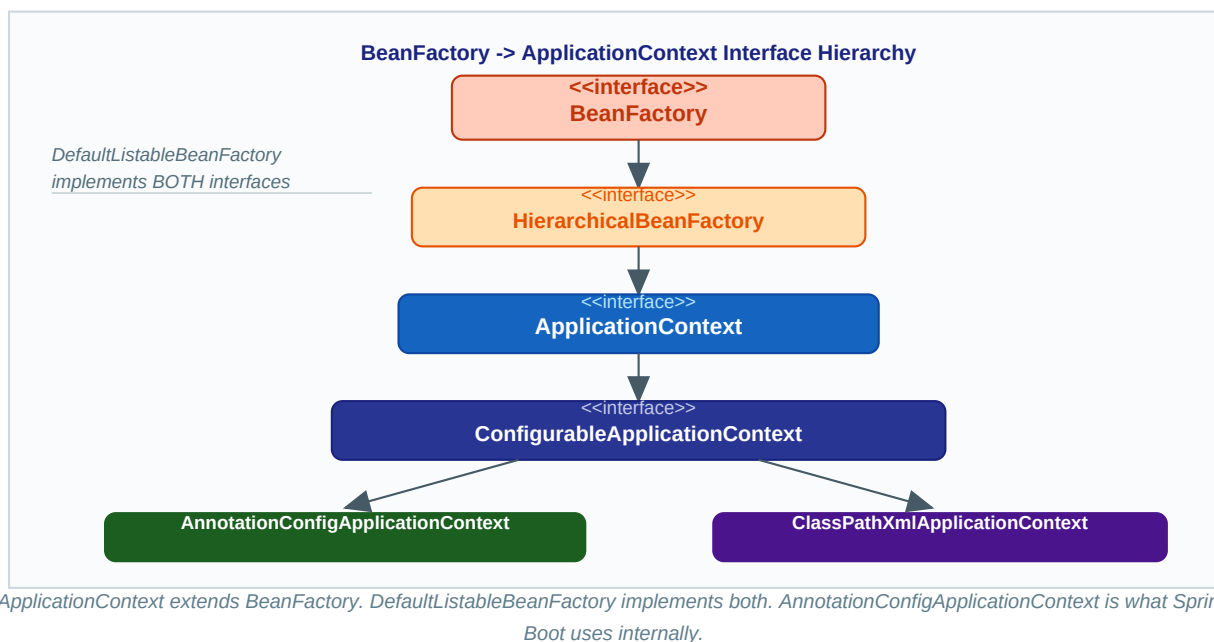
2 Spring Container

ApplicationContext vs BeanFactory

The Spring IoC container is responsible for instantiating, configuring, and assembling beans. Two main interfaces exist: **BeanFactory** (basic, lazy) and **ApplicationContext** (full-featured, eager). In practice, always use ApplicationContext -- BeanFactory alone is only useful in extremely memory-constrained environments or custom framework code.



Interface Hierarchy:



BeanFactory vs ApplicationContext -- detailed comparison:

Feature	BeanFactory	ApplicationContext
Bean loading strategy	Lazy -- bean created on first <code>getBean()</code> call	Eager -- all singletons created at startup
Startup failure detection	Fails only when you call <code>getBean()</code>	Fails fast at startup -- misconfigured beans caught early

AOP support	No automatic AOP proxy creation	Yes -- BeanPostProcessors apply AOP automatically
Application events	Not supported	publishEvent() / ApplicationListener supported
i18n / MessageSource	Not supported	Built-in MessageSource for locale-aware messages
Resource loading	Basic only	ResourceLoader for classpath:, file:, http: URLs
@PostConstruct / @PreDestroy	Not processed	Processed via CommonAnnotation BeanPostProcessor
Environment abstraction	Not available	Environment bean with profiles and properties
Web-aware variant	None	WebApplicationContext for Servlet / Reactive apps
BeanFactoryPostProcessor	Must call manually	Detected and applied automatically
Spring Boot / real apps	Never used	Always used (AnnotationConfigServletWebServerApplicationContext)
Underlying implementation	DefaultListableBeanFactory	Delegates to DefaultListableBeanFactory internally

Code Example 1 -- Raw BeanFactory (shows its limits):

You would almost never write this. It is shown only to understand what ApplicationContext adds on top.

```
import org.springframework.beans.factory.support.DefaultListableBeanFactory;
import org.springframework.beans.factory.support.BeanDefinitionBuilder;

// Rarely done -- only in lightweight/custom framework scenarios
DefaultListableBeanFactory factory = new DefaultListableBeanFactory();

// Manually register a bean definition
factory.registerBeanDefinition("userService",
    BeanDefinitionBuilder
        .rootBeanDefinition(UserService.class)
        .getBeanDefinition());

// BeanFactoryPostProcessors NOT auto-applied -- must call manually
// @PostConstruct NOT processed -- CommonAnnotationBeanPostProcessor missing
// ApplicationEvents NOT available -- no event bus

// Bean is NOT created yet (lazy)
UserService svc = factory.getBean(UserService.class); // created NOW
svc.register("alice");

// Problem: if UserService config is wrong, you find out only at this line
// ApplicationContext would have caught it at startup!
```

Code Example 2 -- ApplicationContext (what you actually use):

```

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Bean;
import org.springframework.core.io.Resource;
import java.util.Locale;

@Configuration
public class AppConfig {
    @Bean
    public UserService userService(UserRepo repo) {
        return new UserService(repo);
    }
}

// Bootstrapping (Spring Boot does this for you)
ApplicationContext ctx =
    new AnnotationConfigApplicationContext(AppConfig.class);
// All singletons are created NOW -- wiring errors caught immediately

// Feature: get a bean
UserService svc = ctx.getBean(UserService.class);

// Feature: publish an event (BeanFactory cannot do this)
ctx.publishEvent(new UserRegisteredEvent(svc, "alice"));

// Feature: load a resource (BeanFactory cannot do this)
Resource res = ctx.getResource("classpath:config/app.yml");

// Feature: read a message for locale (BeanFactory cannot do this)
String msg = ctx.getMessage("welcome.msg", null, Locale.ENGLISH);

```

Code Example 3 -- ApplicationEvent (event publishing in real app):

```

import org.springframework.context.ApplicationEvent;
import org.springframework.stereotype.Service;
import org.springframework.stereotype.Component;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.ApplicationEventPublisher;
import org.springframework.context.event.EventListener;

// Step 1: Define an event
public class UserRegisteredEvent extends ApplicationEvent {
    private final String username;
    public UserRegisteredEvent(Object src, String username) {
        super(src);
        this.username = username;
    }
    public String getUsername() { return username; }
}

// Step 2: Publish from a service
@Service
public class UserService {
    @Autowired
    private ApplicationEventPublisher publisher; // injected by Spring

    public void register(String username) {
        // ... save to DB ...
        publisher.publishEvent(
            new UserRegisteredEvent(this, username));
    }
}

```



```

}

// Step 3: Listen in another bean
@Component
public class EmailSender {
    @EventListener
    public void onRegistered(UserRegisteredEvent evt) {
        System.out.println("Sending welcome email to: "
            + evt.getUsername());
    }
}

// BeanFactory has NO publishEvent() -- this entire flow needs ApplicationContext

```

Behind the scenes -- what happens during `ApplicationContext.refresh()`:

When you call `new AnnotationConfigApplicationContext(AppConfig.class)`, Spring calls **`refresh()`** internally. This is the heart of the container startup:

```

// Simplified view of AbstractApplicationContext.refresh()
// (Spring internal code -- no imports needed, shown for understanding)

// 1. Prepare BeanFactory (DefaultListableBeanFactory)
prepareBeanFactory(beanFactory);

// 2. Apply BeanFactoryPostProcessors (e.g., @PropertySource processing)
invokeBeanFactoryPostProcessors(beanFactory);

// 3. Register BeanPostProcessors (e.g., AutowiredAnnotationBeanPostProcessor)
registerBeanPostProcessors(beanFactory);

// 4. Initialize MessageSource (i18n support)
initMessageSource();

// 5. Initialize ApplicationEventMulticaster (event bus)
initApplicationEventMulticaster();

// 6. Create all singleton beans eagerly
finishBeanFactoryInitialization(beanFactory);

// 7. Publish ContextRefreshedEvent
finishRefresh();

// BeanFactory stops at step 1 -- you get none of steps 2-7 for free

```

Key Rules

- Always use `ApplicationContext` -- `BeanFactory` is too basic for real apps.
- `ApplicationContext.refresh()` does the heavy lifting: post-processors, i18n, events, singletons.
- `DefaultListableBeanFactory` is the actual engine; `ApplicationContext` wraps and enriches it.
- Spring Boot uses `AnnotationConfigServletWebServerApplicationContext` (a subtype) automatically.
- Startup failure is a feature: `ApplicationContext` catches wiring errors before any request comes in.
- `ApplicationEventPublisher` is an extra interface `ApplicationContext` implements -- `BeanFactory` does not.

Q1. What is the Spring IoC container?

It is the runtime that creates and manages bean instances, resolves dependencies, and controls the bean lifecycle.

Q2. What is the key practical difference between `BeanFactory` and `ApplicationContext`?

BeanFactory is lazy and minimal -- it creates beans only on demand and lacks AOP, events, i18n. ApplicationContext is eager (fails fast at startup), processes all post-processors automatically, and supports events, i18n, and resource loading.

Q3. When would you ever use BeanFactory directly?

Almost never. Only in extremely memory-constrained environments (Android, IoT) or when writing custom Spring extensions that need a lightweight container without full ApplicationContext overhead.

Q4. What does ApplicationContext.refresh() do?

It initialises the container: applies BeanFactoryPostProcessors, registers BeanPostProcessors (for @Autowired, AOP, etc.), sets up the event multicaster, and eagerly instantiates all singleton beans.

Q5. What is DefaultListableBeanFactory?

It is the concrete class that actually stores BeanDefinitions and creates beans. Both BeanFactory and ApplicationContext ultimately delegate to it. ApplicationContext wraps it with extra services.

Q6. How does ApplicationContext support events?

It implements ApplicationEventPublisher. You inject ApplicationEventPublisher into any bean, call publishEvent(), and any @EventListener method in any bean receives it asynchronously.

Q7. What is a Spring bean?

Any object managed by the Spring container. Beans are declared via @Component, @Bean, or XML, and Spring instantiates, wires, and destroys them.

Q8. How does Spring Boot configure the container?

Spring Boot auto-detects @SpringBootApplication, scans the package, applies auto-configuration, and bootstraps an AnnotationConfigServletWebServerApplicationContext automatically.

Each comparison point from the table above is explained below with side-by-side code showing what you get (or don't get) with each interface.

1. Bean Loading Strategy: Lazy vs Eager

BeanFactory is lazy: a bean is created only when you explicitly call **getBean()**. If your bean has a broken constructor or missing dependency, you find out only at that call -- potentially deep in production. **ApplicationContext** is eager: all singleton beans are created during **refresh()**. A misconfigured bean fails at startup, not at runtime.

```
import org.springframework.beans.factory.support.DefaultListableBeanFactory;
import org.springframework.beans.factory.support.BeanDefinitionBuilder;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

// --- BeanFactory (LAZY) ---
DefaultListableBeanFactory factory = new DefaultListableBeanFactory();
factory.registerBeanDefinition("orderService",
    BeanDefinitionBuilder.rootBeanDefinition(OrderService.class)
        .getBeanDefinition());

// Bean is NOT created yet -- no error even if OrderService constructor is broken
System.out.println("After registerBeanDefinition -- bean not created yet");

OrderService svc = factory.getBean(OrderService.class); // created HERE
// If OrderService constructor throws -- you crash here, at runtime

// --- ApplicationContext (EAGER) ---
ApplicationContext ctx =
    new AnnotationConfigApplicationContext(AppConfig.class);
// ALL singleton beans created HERE during refresh()
// Broken constructor? App fails to start -- you catch it immediately
// No request ever reaches a broken bean
```

2. Startup Failure Detection (Fail Fast)

With **BeanFactory**, a missing required dependency is invisible until someone requests that bean. With **ApplicationContext**, Spring validates all singleton wiring at startup and throws immediately with a clear error message.

```
import org.springframework.stereotype.Service;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

@Service
public class PaymentService {
    // Requires EmailService -- but imagine no @Service on EmailService
    public PaymentService(EmailService email) { ... }
}

// BeanFactory: No error at registration. Error appears only when:
//   factory.getBean(PaymentService.class) -- called later in code
//   -> NoSuchBeanDefinitionException thrown deep inside business logic

// ApplicationContext:
//   new AnnotationConfigApplicationContext(AppConfig.class)
//   -> UnsatisfiedDependencyException thrown IMMEDIATELY at startup
```

```
// -> Stack trace clearly shows: PaymentService needs EmailService
// -> App does not start -- ops team sees the error in startup logs
// -> No user request ever reaches the broken service
```

3. AOP Support (Aspect-Oriented Programming)

AOP lets you apply cross-cutting behaviour (transactions, security, logging, caching) to beans transparently -- without touching the business code. Spring implements AOP by wrapping beans in **CGLIB proxies** via BeanPostProcessors. **BeanFactory** does not register these post-processors automatically, so AOP annotations are ignored. **ApplicationContext** registers them during refresh().

```
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.cache.annotation.Cacheable;
import org.springframework.beans.factory.support.DefaultListableBeanFactory;
import org.springframework.beans.factory.xml.XmlBeanDefinitionReader;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

@Service
public class OrderService {
    @Transactional          // Begin/commit DB transaction automatically
    public void placeOrder() {
        repo.save(order);    // if exception -> auto rollback
    }

    @Cacheable("orders")    // Cache result in memory
    public Order findOrder(Long id) { return repo.findById(id); }
}

// --- With BeanFactory (AOP DOES NOT WORK) ---
DefaultListableBeanFactory factory = new DefaultListableBeanFactory();
new XmlBeanDefinitionReader(factory).loadBeanDefinitions(...);
OrderService svc = factory.getBean(OrderService.class);
svc.placeOrder();
// @Transactional is IGNORED -- no proxy, no transaction, no rollback
// @Cacheable is IGNORED -- no proxy, method hits DB every time
// You get a plain OrderService object, not a proxy

// --- With ApplicationContext (AOP WORKS) ---
ApplicationContext ctx =
    new AnnotationConfigApplicationContext(AppConfig.class);
OrderService svc2 = ctx.getBean(OrderService.class);
// svc2 is actually OrderService$$SpringCGLIB$$0 (a proxy)
svc2.placeOrder();
// Proxy intercepts the call:
// -> TransactionInterceptor.beginTransaction()
// -> YOUR placeOrder() runs
// -> TransactionInterceptor.commit() or rollback()
```

4. Application Events (publishEvent / @EventListener)

Spring's event system lets beans communicate without knowing about each other. One bean **publishes** an event; any number of other beans **listen** for it. This implements the Observer pattern cleanly. **BeanFactory** has no event bus. **ApplicationContext** implements **ApplicationEventPublisher** and maintains an **ApplicationEventMulticaster** that routes events.

```
import org.springframework.context.ApplicationEvent;
import org.springframework.stereotype.Service;
```

```

import org.springframework.stereotype.Component;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.ApplicationEventPublisher;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.context.event.EventListener;

// Step 1: Define a custom event
public class OrderPlacedEvent extends ApplicationEvent {
    private final Long orderId;
    public OrderPlacedEvent(Object source, Long orderId) {
        super(source);
        this.orderId = orderId;
    }
    public Long getOrderId() { return orderId; }
}

// Step 2: Publisher bean (does not know who listens)
@Service
public class OrderService {
    @Autowired
    private ApplicationEventPublisher publisher;

    @Transactional
    public void placeOrder(Order order) {
        repo.save(order);
        // Fire event -- any listener receives it
        publisher.publishEvent(new OrderPlacedEvent(this, order.getId()));
    }
}

// Step 3: Listener beans (can be in any package, any layer)
@Component
public class EmailNotifier {
    @EventListener
    public void onOrder(OrderPlacedEvent evt) {
        emailService.sendConfirmation(evt.getOrderId());
    }
}

@Component
public class InventoryUpdater {
    @EventListener
    public void onOrder(OrderPlacedEvent evt) {
        inventory.reserve(evt.getOrderId());
    }
}

// BeanFactory: factory.publishEvent() -- METHOD DOES NOT EXIST
// ApplicationContext: ctx.publishEvent(new OrderPlacedEvent(...)) -- works

```

5. Internationalisation (i18n) -- MessageSource

MessageSource resolves locale-specific messages. Useful for error messages, UI labels, and email templates that need to appear in different languages. **BeanFactory** has no MessageSource support. **ApplicationContext** auto-detects a MessageSource bean and makes it available directly via **ctx.getMessage()** or by injecting MessageSource.

```

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.annotation.Configuration;

```

```

import org.springframework.context.annotation.Bean;
import org.springframework.context.MessageSource;
import org.springframework.context.support.ResourceBundleMessageSource;
import java.util.Locale;

// messages_en.properties (on classpath)
//  welcome.msg=Welcome, {0}!
//  error.notfound=Item {0} not found.

// messages_fr.properties
//  welcome.msg=Bienvenue, {0} !
//  error.notfound=Article {0} introuvable.

@Configuration
public class AppConfig {
    @Bean
    public MessageSource messageSource() {
        ResourceBundleMessageSource ms = new ResourceBundleMessageSource();
        ms.setBasename("messages");
        return ms;
    }
}

// ApplicationContext -- works
ApplicationContext ctx =
    new AnnotationConfigApplicationContext(AppConfig.class);

String en = ctx.getMessage("welcome.msg",
    new Object[]{"Alice"}, Locale.ENGLISH);
// -> "Welcome, Alice!"

String fr = ctx.getMessage("welcome.msg",
    new Object[]{"Alice"}, Locale.FRENCH);
// -> "Bienvenue, Alice !"

// BeanFactory -- ctx.getMessage() DOES NOT EXIST on BeanFactory interface
// You would have to manually call messageSource.getMessage() every time

```

6. Resource Loading (classpath:, file:, http:)

ApplicationContext extends **ResourceLoader**, allowing you to load files from the classpath, filesystem, or a URL with a unified API. **BeanFactory** does not implement **ResourceLoader**.

```

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.core.io.Resource;
import org.springframework.core.io.ResourceLoader;
import org.springframework.core.io.ClassPathResource;
import org.springframework.stereotype.Service;
import org.springframework.beans.factory.annotation.Autowired;
import java.io.InputStream;

// ApplicationContext -- resource loading built in
ApplicationContext ctx =
    new AnnotationConfigApplicationContext(AppConfig.class);

// Load from classpath (inside JAR)
Resource r1 = ctx.getResource("classpath:config/settings.yml");

// Load from filesystem
Resource r2 = ctx.getResource("file:/opt/app/overrides.properties");

```

```
// Load from URL
Resource r3 = ctx.getResource("https://config-server/app.json");

// Read content
InputStream in = r1.getInputStream();

// Inject ResourceLoader into any bean
@Service
public class ConfigLoader {
    @Autowired
    private ResourceLoader loader;

    public void load() {
        Resource res = loader.getResource("classpath:config/settings.yml");
        // read res ...
    }
}

// BeanFactory: no getResource() method -- must use ClassPathResource manually
Resource manual = new ClassPathResource("config/settings.yml");
```

7. @PostConstruct and @PreDestroy Lifecycle Callbacks

@PostConstruct marks a method to run after the bean is fully wired -- useful for caches, connection pools, or validation. **@PreDestroy** runs before the bean is removed -- useful for closing connections. **ApplicationContext** processes these via **CommonAnnotationBeanPostProcessor**. **BeanFactory** does not register this post-processor automatically, so both annotations are silently ignored.

```
import jakarta.annotation.PostConstruct; // Spring Boot 3.x (Jakarta EE 9+)
import jakarta.annotation.PreDestroy;   // use javax.annotation.* for Spring Boot 2.x
import org.springframework.stereotype.Service;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.ConfigurableApplicationContext;
import java.util.HashMap;
import java.util.Map;

@Service
public class CacheService {
    private Map<String, String> cache;

    @PostConstruct // runs once, after constructor + @Autowired fields
    public void init() {
        cache = new HashMap<>();
        cache.put("config", loadFromDB()); // pre-warm cache
        System.out.println("Cache warmed up");
    }

    @PreDestroy // runs when ApplicationContext is closed
    public void cleanup() {
        cache.clear();
        System.out.println("Cache cleared, resources released");
    }
}

// ApplicationContext -- @PostConstruct and @PreDestroy WORK
ApplicationContext ctx =
    new AnnotationConfigApplicationContext(AppConfig.class);
// "Cache warmed up" printed here automatically
```

```

((ConfigurableApplicationContext) ctx).close();
// "Cache cleared, resources released" printed here automatically

// BeanFactory -- @PostConstruct and @PreDestroy SILENTLY IGNORED
// init() is never called -- cache is null -- NullPointerException at runtime
// cleanup() is never called -- resources leak

```

8. Environment Abstraction (Profiles and Properties)

ApplicationContext exposes an **Environment** object that provides unified access to system properties, application.properties, and Spring profiles. **BeanFactory** has no Environment abstraction.

```

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.core.env.Environment;
import org.springframework.core.env.Profiles;
import org.springframework.stereotype.Service;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.beans.factory.annotation.Autowired;

// application.properties
//   app.timeout=30
//   spring.profiles.active=prod

// ApplicationContext -- Environment works
ApplicationContext ctx =
    new AnnotationConfigApplicationContext(AppConfig.class);

Environment env = ctx.getEnvironment();
String timeout = env.getProperty("app.timeout");    // "30"
boolean isProd = env.acceptsProfiles(
    Profiles.of("prod"));                          // true

// Inject directly into any bean
@Service
public class OrderService {
    @Value("${app.timeout}")
    private int timeout;

    @Autowired
    private Environment env;

    public void process() {
        if (env.acceptsProfiles(Profiles.of("prod"))) {
            // use stricter validation in prod
        }
    }
}

// BeanFactory: no getEnvironment() method
// @Value injection also does NOT work with plain BeanFactory
// -- PropertySourcesPlaceholderConfigurer is a BeanFactoryPostProcessor
//    that must be applied manually

```

Key Rules

- AOP (@Transactional, @Cacheable) requires ApplicationContext -- BeanFactory gives a plain object.
- @PostConstruct / @PreDestroy are silently ignored by BeanFactory.
- publishEvent() does not exist on BeanFactory -- event-driven design needs ApplicationContext.

- `ctx.getMessage()` (i18n) and `ctx.getResource()` (files) are `ApplicationContext`-only methods.
- `@Value` and `@Profile` both rely on the `Environment` abstraction absent in `BeanFactory`.
- In Spring Boot, the container is always an `ApplicationContext` subtype -- you never touch `BeanFactory`.

Q1. Why does `@Transactional` do nothing with `BeanFactory`?

Spring AOP works by wrapping beans in CGLIB proxies inside `BeanPostProcessors`. `BeanFactory` does not auto-register `BeanPostProcessors` like `AnnotationAwareAspectJAutoProxyCreator`, so no proxy is created and the annotation is silently ignored.

Q2. What is `ApplicationEventPublisher`?

An interface that `ApplicationContext` implements. It lets any bean fire an event (`publishEvent`) that is delivered to all `@EventListener` methods registered in the context.

Q3. What is `MessageSource` used for?

It resolves locale-specific strings from `.properties` files. Used for multi-language UI messages, validation errors, and email templates.

Q4. What happens to `@PostConstruct` with `BeanFactory`?

Nothing -- it is silently ignored. `CommonAnnotationBeanPostProcessor`, which processes `@PostConstruct` and `@PreDestroy`, is not auto-registered with `BeanFactory`.

Q5. What is the `Environment` in `ApplicationContext`?

An abstraction over all property sources (`application.properties`, system env, JVM args) and active profiles. It powers `@Value` injection and `@Profile` bean switching.

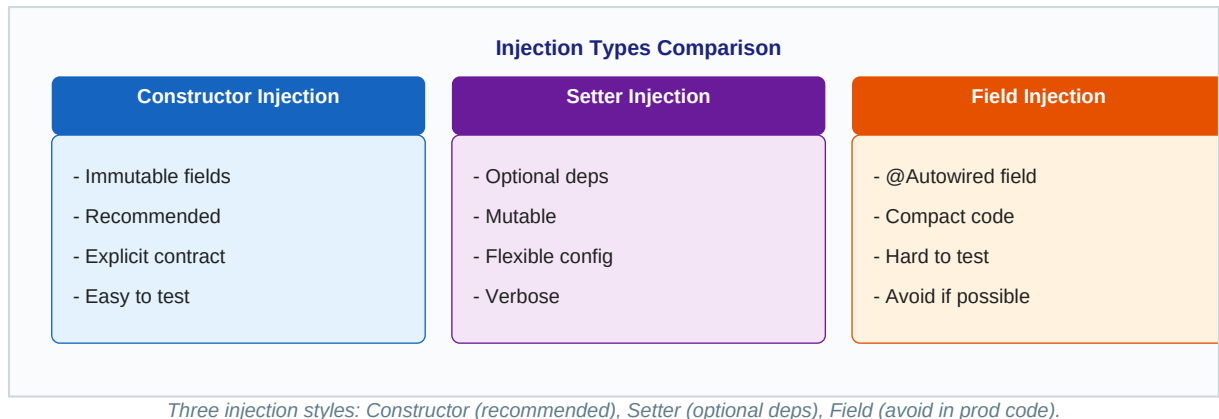
Q6. When would I ever use `BeanFactory` directly?

Almost never in application code. Only in custom Spring extensions, IoT/Android environments, or integration tests where you need an ultra-lightweight container with no extras.

3 Injection Types

Constructor vs Setter vs Field

Spring supports three ways to inject dependencies into a bean. Constructor injection is the recommended approach. Setter injection suits optional dependencies. Field injection is convenient but harder to test.



Constructor Injection (recommended):

```
import org.springframework.stereotype.Service;

@Service
public class UserService {
    private final EmailService email;

    // Spring sees one constructor -- injects automatically
    public UserService(EmailService email) {
        this.email = email;
    }
}
```

Setter Injection (optional dependency):

```
import org.springframework.stereotype.Service;
import org.springframework.beans.factory.annotation.Autowired;

@Service
public class ReportService {
    private LogService log;

    @Autowired // optional -- Spring calls this setter
    public void setLog(LogService log) {
        this.log = log;
    }
}
```

Field Injection (compact but avoid in production):

```
import org.springframework.stereotype.Service;
import org.springframework.beans.factory.annotation.Autowired;

@Service
public class NotifyService {
    @Autowired // Spring injects directly into field
    private SmsService sms;
}
```

Key Rules

- Prefer constructor injection: fields become final, class is immutable.
- If a class has exactly one constructor, `@Autowired` is optional (Spring 4.3+).
- Setter injection is good for optional or reconfigurable dependencies.
- Field injection hides dependencies and makes unit testing harder -- avoid it.
- Circular dependency errors are usually caught at startup with constructor injection.

Q1. Why is constructor injection preferred?

It makes dependencies explicit, allows fields to be final (immutable), and makes the class easy to test without a Spring container.

Q2. When should I use setter injection?

When a dependency is optional or needs to be changed after construction. Mark with `@Autowired(required=false)` if it may be absent.

Q3. What is field injection and why avoid it?

Field injection uses `@Autowired` directly on a field. It hides dependencies, prevents final fields, and requires a running container to test.

Q4. Can Spring detect circular dependencies?

Yes -- with constructor injection it throws a `BeanCurrentlyInCreationException` at startup. Setter injection may hide circular deps, causing runtime errors.

Q5. Do I need `@Autowired` on a single constructor?

No. Since Spring 4.3, if a class has exactly one constructor Spring injects it automatically without `@Autowired`.

4 Stereotype Annotations

@Component, @Service, @Repository, @Controller

Spring provides four stereotype annotations that mark a class as a Spring-managed bean. They are all specialisations of **@Component**, but carry semantic meaning and enable extra features (e.g., exception translation for **@Repository**).

Annotation	Layer	Extra behaviour
@Component	Generic	Base annotation -- no extras
@Service	Business logic	Signals service layer -- no extras
@Repository	Data access (DAO)	Translates SQL exceptions to Spring DataAccessException
@Controller	Web (Spring MVC)	Returns a VIEW NAME -- Spring resolves to HTML template
@RestController	Web (REST API)	@Controller + @ResponseBody -- writes JSON/XML directly to HTTP response

@Component -- Generic utility bean:

Use when your class does not belong to any specific layer -- e.g. a helper, converter, scheduler, or custom validator. It is the base annotation all others are built on.

```
import org.springframework.stereotype.Component;
import org.springframework.beans.factory.annotation.Autowired;

// Generic helper -- not a service, not a repo, not a controller
@Component
public class CurrencyConverter {
    public double usdToInr(double usd) {
        return usd * 83.5; // just a utility
    }
}

// Any other bean can inject it
@Component
public class InvoicePrinter {
    private final CurrencyConverter converter;

    @Autowired
    public InvoicePrinter(CurrencyConverter converter) {
        this.converter = converter;
    }

    public void print(double usd) {
        System.out.println("INR: " + converter.usdToInr(usd));
    }
}
```

@Service -- Business logic layer:

@Service is functionally identical to @Component but signals that this class contains business rules. It sits between the controller (web) and the repository (DB).

```

import org.springframework.stereotype.Service;
import org.springframework.beans.factory.annotation.Autowired;

@Service
public class OrderService {
    private final OrderRepository repo;
    private final EmailService emailService;

    // Constructor injection -- both dependencies are mandatory
    public OrderService(OrderRepository repo, EmailService emailService) {
        this.repo = repo;
        this.emailService = emailService;
    }

    // Business rule: total must be positive, notify after saving
    public Order placeOrder(Order order) {
        if (order.getTotal() <= 0) {
            throw new IllegalArgumentException("Order total must be positive");
        }
        Order saved = repo.save(order);
        emailService.sendConfirmation(saved.getId());
        return saved;
    }
}
// OrderService knows nothing about HTTP -- that is the controller's job
// OrderService knows nothing about SQL -- that is the repository's job

```

@Repository -- Data access layer with exception translation:

The key extra behaviour: Spring wraps any low-level database exception (e.g. **SQLException**, **HibernateException**) into a unified **DataAccessException** hierarchy. Your service layer never needs to know whether you are using JDBC, Hibernate, or JPA.

```

import org.springframework.stereotype.Repository;
import org.springframework.dao.DataAccessException;
import org.springframework.dao.EmptyResultDataAccessException;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.beans.factory.annotation.Autowired;

@Repository
public class OrderRepository {
    private final JdbcTemplate jdbc;

    public OrderRepository(JdbcTemplate jdbc) {
        this.jdbc = jdbc;
    }

    public Order findById(Long id) {
        // JdbcTemplate throws DataAccessException (Spring)
        // NOT raw SQLException (JDBC)
        return jdbc.queryForObject(
            "SELECT * FROM orders WHERE id = ?",
            new Object[]{id},
            (rs, row) -> new Order(rs.getLong("id"), rs.getString("status"))
        );
    }
}

// Without @Repository -- raw JDBC example of what you get:
// java.sql.SQLException: No data found <-- JDBC specific
//
// With @Repository -- Spring translates to:

```

```
// EmptyResultDataAccessException      <-- Spring unified, DB-agnostic
//
// Service layer catches DataAccessException -- works whether
// you switch from MySQL to PostgreSQL to MongoDB underneath
```

@Controller -- Returns an HTML view name (NOT data):

@Controller is for **server-side rendered** web applications (Thymeleaf, JSP). The method returns a **String** which is the name of an HTML template. Spring MVC finds that template, merges it with model data, and sends back HTML to the browser.

```
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.beans.factory.annotation.Autowired;

@Controller          // NOT @RestController
public class OrderController {
    private final OrderService orderService;

    public OrderController(OrderService orderService) {
        this.orderService = orderService;
    }

    @GetMapping("/orders/{id}")
    public String showOrder(@PathVariable Long id, Model model) {
        Order order = orderService.findById(id);

        // Put data into the model -- Thymeleaf template reads it
        model.addAttribute("order", order);

        // Return the VIEW NAME -- NOT the data itself
        return "order-detail"; // Spring looks for: templates/order-detail.html
    }
}

// Browser sends:  GET /orders/42
// Spring calls:   showOrder(42, model)
// Method returns: "order-detail"    <-- just a String, not JSON
// Spring finds:   src/main/resources/templates/order-detail.html
// Thymeleaf fills template with model.order
// Browser receives: full HTML page
```

Thymeleaf template (order-detail.html) that @Controller renders:

```
<!-- src/main/resources/templates/order-detail.html -->
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>
    <h1>Order Details</h1>
    <!-- th:text reads from model.addAttribute("order", order) -->
    <p>Order ID:      <span th:text="${order.id}"></span></p>
    <p>Order Status: <span th:text="${order.status}"></span></p>
    <p>Order Total:  <span th:text="${order.total}"></span></p>
</body>
</html>

<!-- Spring renders this to HTML and sends it to the browser -->
<!-- The browser displays a webpage, not raw JSON -->
```

@RestController -- Returns JSON/XML data directly (NOT a view):

@RestController = **@Controller** + **@ResponseBody** on every method. The return value is **serialized to JSON** (via Jackson) and written directly into the HTTP response body. There is no template, no HTML -- just raw data. Used for REST APIs consumed by mobile apps, frontends, or other services.

```
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.http.ResponseEntity;
import org.springframework.http.HttpStatus;

@RestController // NOT @Controller
public class OrderRestController {
    private final OrderService orderService;

    public OrderRestController(OrderService orderService) {
        this.orderService = orderService;
    }

    @GetMapping("/api/orders/{id}")
    public Order getOrder(@PathVariable Long id) {
        return orderService.findById(id);
        // Jackson converts Order object -> JSON automatically
        // No "return view name" -- the object IS the response
    }

    @PostMapping("/api/orders")
    public ResponseEntity<Order> createOrder(@RequestBody Order order) {
        Order saved = orderService.placeOrder(order);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }
}

// Client sends: GET /api/orders/42
// Spring calls: getOrder(42)
// Method returns: Order object
// Jackson converts to JSON:
// {
//   "id": 42,
//   "status": "CONFIRMED",
//   "total": 1500.00
// }
// Browser / mobile app receives: pure JSON -- no HTML at all
```

@Controller vs @RestController -- key difference side by side:

Aspect	@Controller	@RestController
Returns	String (view name) or ModelAndView	Java object serialized to JSON/XML
Response body	HTML page rendered from template	Raw JSON or XML data
Used with	Thymeleaf, JSP (server-side HTML)	React, Angular, mobile apps, other APIs
@ResponseBody needed?	Yes, on each method if returning data	No -- applied automatically to all methods

URL convention	/orders/42 (browser-friendly)	/api/orders/42 (API endpoint)
Who consumes it	Web browser	Any HTTP client (frontend, Postman, mobile)

Key Rules

- @Component is the root -- @Service, @Repository, @Controller all extend it.
- @Service and @Component are functionally identical -- @Service adds readability.
- @Repository adds exception translation: raw SQLException becomes DataAccessException.
- @Controller returns a view name -- Spring resolves it to an HTML template.
- @RestController returns data -- Jackson serializes the object to JSON automatically.
- Never mix @Controller and @RestController in the same class -- pick one.

Q1. What is the difference between @Component, @Service, and @Repository?

Functionally they are the same (all register a bean). @Service adds semantic meaning. @Repository also enables exception translation from raw DB exceptions to Spring DataAccessException.

Q2. What does @Repository exception translation actually do?

It wraps driver-specific exceptions (SQLException, HibernateException) into Spring's DataAccessException hierarchy. Your service layer stays decoupled from the underlying DB technology.

Q3. What is the difference between @Controller and @RestController?

@Controller returns a view name that Spring resolves to an HTML template. @RestController returns a Java object that Jackson serializes to JSON and writes directly to the HTTP response body.

Q4. Can I use @Controller to return JSON?

Yes -- add @ResponseBody to each individual method. @RestController is just a shortcut that applies @ResponseBody to ALL methods automatically.

Q5. What is the Model in @Controller?

A map of key-value pairs passed to the template. model.addAttribute(key, value) makes the value available as \${key} in Thymeleaf/JSP templates.

Q6. How does Spring find beans marked with stereotypes?

Via component scanning. @SpringBootApplication enables scanning of the package and all sub-packages automatically.

5 @Autowired, @Qualifier, @Primary

Wiring beans when multiple candidates exist

When Spring wires a dependency it looks for a matching bean by type. If multiple beans of the same type exist, it throws a **NoUniqueBeanDefinitionException**. **@Qualifier** picks a specific bean by name; **@Primary** marks the default choice.

Java Example -- @Autowired with @Qualifier:

```
import org.springframework.stereotype.Service;
import org.springframework.beans.factory.annotation.Qualifier;

public interface NotificationService { void send(String msg); }

@Service("emailNotif")
public class EmailNotif implements NotificationService { ... }

@Service("smsNotif")
public class SmsNotif implements NotificationService { ... }

@Service
public class AlertService {
    private final NotificationService notif;

    // Tell Spring which implementation to inject
    public AlertService(
        @Qualifier("emailNotif") NotificationService notif) {
        this.notif = notif;
    }
}
```

Java Example -- @Primary (default bean):

```
import org.springframework.stereotype.Service;
import org.springframework.context.annotation.Primary;
import org.springframework.beans.factory.annotation.Autowired;

@Primary // used when no @Qualifier is specified
@Service("emailNotif")
public class EmailNotif implements NotificationService { ... }

@Service("smsNotif")
public class SmsNotif implements NotificationService { ... }

// Spring picks EmailNotif automatically -- it is @Primary
@Autowired
private NotificationService notif;
```

Key Rules

- **@Autowired** tells Spring to inject a matching bean by type.
- If multiple beans match, use **@Qualifier("beanName")** to pick one.
- **@Primary** marks the default bean when no **@Qualifier** is present.
- **@Qualifier** overrides **@Primary** -- it is more specific.
- Since Spring 4.3, constructor injection does not need **@Autowired**.

Q1. What happens when Spring finds two beans of the same type?

It throws **NoUniqueBeanDefinitionException** at startup. Resolve it with **@Qualifier** or **@Primary**.

Q2. What is @Qualifier?

@Qualifier("name") on an injection point tells Spring which specific bean (by name) to inject when multiple candidates exist.

Q3. What is @Primary?

@Primary on a bean definition marks it as the default choice. Spring uses it automatically when no @Qualifier is specified.

Q4. Which takes priority -- @Primary or @Qualifier?

@Qualifier always wins. @Primary is only used when no explicit qualifier is present.

Q5. Can @Autowired be used on methods?

Yes -- @Autowired works on constructors, setter methods, and fields. Spring will call the method and inject all parameters.

6 @Configuration and @Bean

Java-based container configuration

Instead of XML, you can configure Spring beans with plain Java using **@Configuration** classes. Each **@Bean** method returns an object that Spring registers as a managed bean. This is type-safe, IDE-friendly, and refactorable.

Java Example -- @Configuration class:

```
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Bean;

@Configuration
public class AppConfig {

    @Bean                // registers as a bean named 'userRepo'
    public UserRepo userRepo() {
        return new UserRepo();
    }

    @Bean                // Spring injects userRepo() automatically
    public UserService userService(UserRepo repo) {
        return new UserService(repo);
    }
}
```

Java Example -- using the config:

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

ApplicationContext ctx =
    new AnnotationConfigApplicationContext(AppConfig.class);

UserService svc = ctx.getBean(UserService.class);
svc.createUser("bob");

// Spring Boot discovers @Configuration classes automatically
```

@Configuration vs @Component -- summary:

Aspect	@Configuration + @Bean	@Component + @Autowired
Style	Explicit factory methods	Auto-detected via scan
Use case	Third-party classes, custom init logic	Your own classes
Proxy	@Bean methods are CGLIB-proxied (singleton safe)	No proxy -- direct instantiation
Readability	All wiring in one place	Wiring spread across classes

Key Rules

- @Configuration marks a class as a source of @Bean definitions.
- @Bean methods are like factory methods -- Spring calls them once and caches the result.
- Use @Configuration for third-party classes you cannot annotate yourself.
- @Bean method parameters are auto-wired by Spring -- no @Autowired needed.

- Spring Boot's `@SpringBootApplication` includes `@EnableAutoConfiguration` which processes `@Configuration` classes.

Q1. What is `@Configuration`?

It marks a class as a Spring configuration class containing `@Bean` methods. Spring processes it like a factory for beans.

Q2. What is `@Bean`?

`@Bean` on a method tells Spring to manage the returned object as a bean. The method name becomes the default bean name.

Q3. When should I use `@Configuration`/`@Bean` instead of `@Component`?

Use it for third-party library classes (which you cannot annotate), or when you need fine-grained control over construction.

Q4. Are `@Bean` methods called multiple times?

No. Spring uses CGLIB proxying to intercept calls and return the same singleton instance every time a `@Bean` method is called.

Q5. Can a `@Bean` method receive parameters?

Yes. Spring automatically injects beans matching the parameter types -- no `@Autowired` annotation required on the parameters.